

CCUT 현재판 바이블

부제

채팅형 편집 제안 프로그램으로 재정렬된 CCUT의 완전 정의서

이 문서의 목적

이 문서는 현재 시점의 CCUT을 처음부터 끝까지 다시 복원하기 위한 **완전 기준 문서**다.

이 문서를 다시 읽으면, 이전 대화가 사라져도 아래를 즉시 이해할 수 있어야 한다.

- 지금 CCUT이 무엇인지
- 왜 이전의 복잡한 편집기 형태를 버렸는지
- 왜 채팅형 구조로 돌아왔는지
- A/B 연속 제안이 왜 핵심인지
- 우측창은 왜 편집창이 아니라 구조맵인지
- 좌/중/우 역할이 어떻게 다시 정의되었는지
- Archive / Export / Design이 어떤 위치로 내려갔는지
- 여러 영상을 받았을 때 왜 하나의 작업공간으로 묶는지
- 코덱스가 만든 안정 구조 중 무엇을 살렸는지
- 사용자가 왜 “배워야 하는 툴”을 싫어하는지
- 출시 전까지 어떤 기준으로 판단해야 하는지

이 문서는 단순 메모가 아니다.

지금의 CCUT을 다시 세우기 위한 **제품 바이블**이다.

PART 1 . 왜 방향을 바꿨는가

1 . 1 출발점

CCUT은 오랫동안 “강한 편집 도구”를 지향해 왔다.

그 과정에서 아래 요소들이 붙었다.

- 좌측 프로젝트/메뉴
- 중앙 proposal 및 상태 카드
- 우측 source/edit/board 성격의 패널
- drag/drop 느낌의 조작 흐름
- archive/export/design 등 관리 화면
- direct editing 흔적

이 구조는 기술적으로는 풍부했지만, 제품으로 봤을 때 큰 문제가 있었다.

핵심 문제

보는 순간 배워야 할 툴처럼 보였다.

즉, CCUT은 원래 “바로 옆에 두고 쓰는 존재”가 되어야 하는데, 어느 순간 “공부해서 써야 하는 편집기”가 시작했다.

1.2 사용자가 왜 도망가는가

사용자는 첫 화면을 보는 순간 아래를 판단한다.

- 뭘 먼저 해야 하는지 보이는가
- 내가 이걸 배워야 하는가
- 지금 당장 쓸 수 있는가
- 복잡한가, 단순한가

기존 대시보드형 CCUT은 첫인상에서 아래처럼 읽혔다.

- 패널이 많음
- 카드가 많음
- 상태 정보가 많음
- 조작점이 많음
- 편집기처럼 보임

결과적으로 사용자는 이렇게 느낀다.

“좋아 보이는데, 배우기 귀찮다.”

이 순간 80%는 떠난다.

1.3 ChatGPT와 같은 구조를 지향하게 된 이유

사용자는 ChatGPT를 배워서 쓰지 않는다.

그냥 연다. 입력한다. 결과를 받는다.

이 구조가 강한 이유는 단순하다.

- 기본은 하나의 중심 영역뿐이다.
- 해야 할 행동이 명확하다.
- 나머지는 숨어 있다.
- 필요하면 깊게 들어갈 수 있지만, 기본은 가볍다.

CCUT도 이 구조를 닮아야 한다는 결론에 도달했다.

즉:

- 기본은 채팅형 인터페이스
- 사용자는 영상 업로드
- 시스템은 분석
- Proposal A/B 제안
- 사용자는 선택
- 다시 Proposal A/B
- 관리/기록/수출은 뒤쪽 보조 흐름
- 우측창은 필요할 때만 여는 구조맵

PART 2 . 현재 CCUT의 최종 정의

2 . 1 제품 한 문장 정의

CCUT은 채팅형 편집 제안 프로그램이다. 사용자는 영상을 업로드하고, CCUT은 이를 분석해 두 개의 러프컷 B)을 제안하며, 사용자는 선택·재제안·기록·내보내기를 통해 빠르게 usable rough cut을 얻는다.

2 . 2 더 짧은 정의

기본은 채팅, 핵심은 A/B 제안, 보조는 우측 구조맵, 뒤에는 기록과 수출.

2 . 3 이것은 무엇이 아닌가

현재 CCUT은 다음이 아니다.

- 직접 편집기
- 프리미어 대체 툴
- drag/drop 중심 편집기
- 대시보드형 관리도구
- board/trash 조합 툴
- 텍스트 명령으로 모든 것을 편집하는 도구
- 복잡한 전문가용 NLE

즉, 사용자는 CCUT 안에서 “세밀한 편집 조작”을 하는 것이 아니라, 구조를 보고, 제안을 받고, 고르고, 관리한다.

PART 3 . 핵심 철학

3 . 1 배워야 하는 툴이 되면 실패

가장 중요한 철학은 이것이다.

배워야 하는 툴이 되면 사용자는 떠난다.

CCUT은 설명서형 제품이 아니라, 바로 사용할 수 있는 제품이어야 한다.

그래서 다음이 중요하다.

- 첫 화면에서 행동이 보여야 함
- 사용자가 메뉴를 공부하면 안 됨
- 기능은 많아도 기본은 단순해야 함
- 깊이는 숨겨져 있어야 함

3.2 편집 공장이라는 관점

현재 CCUT의 정체성은 “방송국 편집기”가 아니라 **편집 공장**에 가깝다.

공장의 핵심은 아래와.

- 입력
- 처리
- 제안
- 선택
- 반복
- 결과물
- 기록

즉, 사용자는 복잡한 수동 조작자가 아니라, **입력과 선택을 반복하는 관리자**에 가깝다.

3.3 A/B 연속 제안이 중심

CCUT의 핵심은 항상 두 가지 안을 제안하는 것이다.

- Proposal A
- Proposal B

사용자는 둘 중 하나를 고른다.

그러면 시스템은 그 선택을 기준으로 다시 두 안을 제안한다.

즉 구조는:

1. 업로드
2. A/B
3. 선택
4. 다시 A/B
5. 선택
6. export / archive

이 흐름이 CCUT의 중심이다.

PART 4 . 현재 화면 구조

현재 CCUT은 겉으로는 훨씬 단순해졌지만, 내부적으로는 여전히 안정된 상태 구조를 유지한다.

4 . 1 전체 화면 철학

기본

- 중앙 채팅창만이 주인공이다.

보조

- 좌측은 얇은 프로젝트 서랍이다.
- 우측은 숨겨진 구조맵이다.

뒤쪽

- Archive / Export / History는 보조 흐름이다.

즉, 사용자는 처음부터 3 단 편집기 구조를 보지 않는다.

4 . 2 좌측

역할

- 프로젝트 전환
- 새 작업 시작
- 업로드 진입
- Archive / Export 진입
- 구조맵 토글

의도

좌측은 또 하나의 대시보드가 아니라 **조용한 서랍**이어야 한다.

하지 않는 것

- 제품 설명 과다
 - 카드형 정보 요약
 - 사용자가 공부해야 하는 메뉴 구조
-

4 . 3 중앙

역할

중앙은 항상 **현재 해야 할 일** 하나만 보여준다.

상태별 예시는 아래와 같다.

아무 것도 없는 상태

- 영상 업로드 유도
- 최근 작업 이어가기

업로드 직후

- 분석 중 메시지
- 진행 상태

분석 완료

- Proposal A
- Proposal B
- 제안 설명
- 선택 버튼

선택 후

- 선택 확인
- 재제안 유도
- export 진입 유도

즉 중앙은 대시보드가 아니라 **대화 흐름**이다.

4 . 4 우측

현재 정의

우측창은 편집창이 아니다.

우측창은 **구조맵**이다.

특징

- 기본은 닫혀 있음
- 사용자가 원할 때만 열림
- source row 표시
- fragment strip 표시
- Proposal A/B 사용 구간 표시
- active version overlap 표시

비유

F 1 2 처럼, 필요할 때만 여는 구조 확인 창

즉, 메인 무대가 아니라 보조 분석 도구다.

PART 5 . 우측 구조맵 상세 정의

5 . 1 왜 버리지 않았는가

우측창은 지금까지 만들어진 자산 중 중요한 것 중 하나다.
완전히 버릴 이유는 없다.

왜냐하면 이것은 단순한 UI 패널이 아니라,

- source 구조를 보여주고
- 제안이 무엇을 썼는지 보여주고
- 사용자가 제안의 근거를 이해하게 하는

설명용 지도이기 때문이다.

5 . 2 왜 편집창으로 두지 않는가

문제는 우측창이 편집창이 되는 순간 다시 모든 것이 무거워진다는 점이다.

- drag/drop
- 길이 조절
- 위치 변경
- board/trash
- 조작 affordance 증가

이것이 살아나는 순간 CCUT은 다시 배워야 하는 편집기로 돌아간다.

그래서 현재 판단은 다음과 같다.

유지할 것

- 구조 보기
- source row
- proposal overlay
- active version 표시

버릴 것

- 직접 편집 조작
- 편집 affordance

- 전면 노출
-

5.3 이상적인 사용 방식

우측창은 이렇게 쓰인다.

- 기본은 닫혀 있다.
- 사용자가 “구조가 궁금하다”고 느낄 때 연다.
- A/B가 source 어디를 쓰는지 본다.
- active version이 어떤 구조인지 본다.
- 확인 후 다시 닫는다.

즉, 우측창은 기본 흐름을 방해하지 않는다.

PART 6 . Proposal A/B 정의

6.1 canonical source

현재 Proposal 생성의 진실은 backend canonical route로 고정되어 있다.

핵심 경로: - `proposal_engine.py` - `proposal_api.py`

프론트는 proposal을 만들지 않고, 소비만 한다.

6.2 Proposal A

정의

안정형 제안

특징

- 더 긴 조각 위주
- 더 적은 컷 수
- 대표 장면 중심
- 무난한 opening
- 안정적인 pacing

사용자 인상

“아, 이걸 더 안전하고 정리된 안이네.”

6 . 3 Proposal B

정의

탐색형 제안

특징

- 더 짧은 조각
- 더 많은 컷 수
- 빠른 전환
- 다른 opening bias
- 낮은 overlap
- 더 실험적인 흐름

사용자 인상

“아, 이건 좀 더 빠르고 자극적이네.”

6 . 4 성공 기준

사용자는 업로드 후 30 초 안에 다음을 느껴야 한다.

- A와 B가 분명히 다르다.
 - 둘 중 하나를 선택할 이유가 있다.
 - 이 틀은 나 대신 방향을 제안해준다.
-

PART 7 . 여러 영상파일을 받을 때의 규칙

이 부분은 중요하다.

CCUT은 영상 1 개 = 프로젝트 1 개로 보지 않는다.

7 . 1 원칙

여러 영상파일은 하나의 작업공간에 들어가고, 제안은 항상 통합 A/B 두 개만 만든다.

즉: - source는 여러 개 - workspace는 하나 - proposal은 두 개

7 . 2 source 보관 방식

원본은 source별로 분리 보관한다.

- source 1

- source 2
- source 3

각 source는 구조맵에서 한 줄씩 가진다.

즉 우측 구조맵에서는: - source 1 → row 1 - source 2 → row 2 - source 3 → row 3

7.3 proposal 생성 방식

proposal은 전체 source를 통합한 fragment pool을 바탕으로 A/B 두 개만 생성한다.

금지: - source마다 proposal 만들기 - 4개, 6개, 8개 proposal 보여주기 - 사용자에게 조합을 계산해 만들기

즉: 원본은 분리 보관, 제안은 통합 생성, 사용자는 A/B만 비교

7.4 기존 작업에 source를 추가할 때

사용자에게 선택권을 준다.

질문

이 영상 묶음을 어떻게 반영할까요?

- 현재 작업에 원본만 추가
- 현재 작업 기준 제안 2개 다시 받기
- 새 작업으로 시작

이 구조가 멀티소스를 단순하게 만든다.

PART 8. Archive와 Export의 위치

8.1 Archive

Archive는 제품의 전면 메인이 아니다.

Archive는 기록 참고다.

여기에 남는 것: - upload - proposal-generated - proposal-selected - regenerate - export

즉, Archive는 사용자의 결정을 시간축으로 저장한다.

8.2 Export

Export는 현재 active version 기준의 handoff 단계다.

현재 v 1 기준 export는 다음과 같은 external-finish package에 가깝다.

- manifest JSON
- edit decision CSV
- README TXT

즉, 완전한 flattened mp 4 export가 아니라 **외부 마감용 패키지**다.

이건 현재 v 1 의 범위 고정 결과다.

8.3 왜 보조 흐름인가

Archive와 Export는 중요하지만, 첫 화면의 주인공이 되면 안 된다.
그렇게 되면 제품이 다시 관리 틀처럼 보인다.

그래서 위치는 다음과 같다.

- 메인: 중앙 채팅 흐름
 - 보조: Archive / Export
-

PART 9. 코덱스가 만든 내부 안정 구조

현재 CCUT의 좋은 점은, 겉모습은 다시 단순해졌지만 내부는 코덱스가 꽤 안정화했다는 것이다.

유지된 내부 자산

- `WorkspaceContext` 의 workspace/source/proposal/history SSOT
- backend canonical Proposal A/B 흐름
- `/generate-fragments` → `/generate-proposals` 런타임 경로
- versionHistory / archiveEntries / exportHistory 연결
- external-finish package export 흐름

즉, 겉은 기존 CCUT에 가깝게 가되, 속은 훨씬 정리됐다.

PART 10. 왜 “겉은 기존 CCUT, 속은 안정 구조”인가

이 판단이 현재의 핵심이다.

걸을 기존 CCUT 쪽으로 되돌린 이유

- 채팅 중심이 더 쉽다
- 덜 무섭다
- 배울 필요가 적다
- 바로 행동하게 만든다

속을 코덱스 구조로 유지한 이유

- A/B canonicalization이 정리됐다
- archive/history/export가 연결됐다
- workspace 생성/삭제/복구가 안정화됐다
- backend 검증 경로가 만들어졌다

즉: 걸은 단순화, 속은 안정화

이것이 지금 CCUT의 가장 중요한 성취다.

PART 1 1 . Keep / Revise / Drop 정리

Keep

- 중앙 Proposal A/B 영역
- 좌측 프로젝트 목록
- Right Panel의 구조맵 기능 자체
- backend proposal canonical flow
- workspace/history/export SSOT
- archive 기록 구조
- export package 흐름

Revise

- 우측창: 기본 숨김 + 구조맵으로 사용
- 좌측: 더 조용한 프로젝트 서랍
- 업로드 유도: 대화형 시작
- Archive/Export: 보조 흐름으로 정리
- 중앙 제안 블록: 채팅형 표현 강화

Drop

- direct editing
- drag/drop editing
- board/trash
- Proposal C/D
- 대시보드형 첫 화면
- 카드가 많은 Home/Archive 전면 구조

- chat 자유문장 편집 명령
- 큰 raw player

Hold

- durable media storage
 - flattened mp 4 export
 - split-panel 잔재 정리
 - 시스템 Python/packaging 최종 배포 방식
-

PART 1 2 . 현재 사용 흐름

기본 사용자 흐름

- 1 . 프로젝트 진입
- 2 . 영상 업로드
- 3 . fragment 생성
- 4 . Proposal A / B 표시
- 5 . 사용자 선택
- 6 . regenerate로 다시 A / B
- 7 . 필요하면 우측 구조맵 열기
- 8 . export 또는 archive 확인

이 흐름은 최대한 배우지 않고도 자연스럽게 이해되어야 한다.

첫 화면에서 사용자에게 보여야 할 것

- 영상 올리기
- 러프컷 2 안 받기
- 현재 작업 이어가기

즉, 사용자는 첫 5 초 안에 다음을 알아야 한다.

“아, 영상 올리면 두 가지 안을 제안해주는구나.”

PART 1 3 . 사용자 인상 관리

1 3 . 1 첫인상

현재 목표는 사용자가 이렇게 느끼게 만드는 것이다.

- 어렵지 않다
- 바로 시작할 수 있다

- 두 가지 안을 비교해볼 수 있다
- 구조가 궁금하면 더 깊게 볼 수 있다

1 3 . 2 절대 피해야 할 인상

- 전문가용 편집기 같다
- 배워야 할 것 같다
- 화면이 너무 많다
- 공부해야 한다
- 어디를 눌러야 할지 모르겠다

PART 1 4 . 지금 남아 있는 위험

1 4 . 1 기술적 위험

- objectUrl 기반 persistence 한계
- flattened mp 4 export 없음
- split-panel 관련 옛 파일이 저장소에 일부 남음
- 환경에 따라 Python packaging 정리가 추가로 필요

1 4 . 2 제품적 위험

- 중앙 제안 블록이 다시 카드형 대시보드처럼 무거워질 수 있음
- Archive/Export가 다시 전면 주연이 될 수 있음
- 우측 구조맵이 다시 편집창처럼 보일 수 있음
- 좌측이 다시 설명이 많은 툴 사이드바로 무거워질 수 있음

PART 1 5 . 앞으로의 polish 원칙

현재 큰 구조는 유지한다.

이제 필요한 건 구조를 다시 흔드는 것이 아니라 **polish**다.

polish의 방향

- 첫 화면의 행동 유도 문구를 짧게
- 중앙 채팅 흐름의 자연스러움 강화
- 좌측의 조용함 유지
- 우측 구조맵의 가독성 개선
- 설명보다 행동을 강조

즉, 더 많이 만드는 것이 아니라 더 덜 무섭게 만드는 단계다.

PART 1 6 . 코덱스/안티그래비티 같은 에이전트에게 줄 원칙

1 6 . 1 항상 먼저 시켜야 할 것

- 지금은 만들기보다 고르기
- 읽기 전용 감사
- keep / revise / drop / hold 분류
- 제품 철학과 맞는지 판정

1 6 . 2 구현 전에 물어야 할 것

- 이게 더 쉬워지는가
- 채팅 중심 구조를 해치지 않는가
- 사용자가 배워야 할 것처럼 보이지 않는가
- Right Panel을 편집창으로 되돌리지 않는가

1 6 . 3 항상 넣어야 할 금지 문구

- direct editing 복구 금지
- Right Panel 플레이어화 금지
- Proposal C/D 금지
- board/trash 복구 금지
- 대시보드형 Home 복구 금지
- 기능 확장 금지

PART 1 7 . 최종 한 문장

현재 CCUT은 채팅형 편집 제안 프로그램이며, 사용자는 영상을 업로드하고 A/B 러프컷 제안을 받으며, 필요할 때 구조맵을 열어 구조를 확인하고, 선택·재제안·기록·내보내기를 통해 usable rough cut을 빠르게 만드는 시

다.

부록 A. 아주 짧은 기억 복원용 요약

CCUT은 무엇인가?

채팅형 편집 제안 프로그램

중심은 무엇인가?

영상 업로드 → Proposal A/B → 선택 → regenerate

우측창은 무엇인가?

기본 숨김 구조맵

Archive는 무엇인가?

기록 참고

Export는 무엇인가?

active version handoff

여러 영상은 어떻게 처리하는가?

하나의 workspace 안에 source별로 넣고, proposal은 A/B 두 개만 통합 생성

버린 것은?

direct editing, board, C/D, 대시보드형 첫 화면, 큰 raw player

유지한 것은?

A/B canonical backend 흐름, WorkspaceContext SSOT, archive/history/export 연결

가장 중요한 철학은?

배워야 하는 툴이 되면 실패한다